带因果的结构 IR

容器面与状态面：提升 AI 生成 RTL 的能力，实现可追溯的 AI 验证

组会汇报 · 2026-11

背景 ①：模拟器把“指令”和“部件”的关系藏在了代码里

硬件实际工作时：**指令是灵魂，部件是身体**——指令依次流过各个部件，部件的位置和状态决定指令哪一拍能走

模拟器	组织方式
GPGPU-Sim / Accel-Sim	GPU 周期级模拟器； route B 用它的 FML 移植版
gem5 （ MinorCPU / O3CPU ）	最常用的 CPU 模拟器
CVA6 官方性能模型	Python ，约 670 行；我们的实验对象
curryGPU 性能模型	按指令类给延迟 / 吞吐，按等待原因统计停顿
PyMTL3	功能级 / 周期级 / RTL 三层，周期级与 RTL 不要求一一对应

以 CVA6 模型为例（我们逐条核对过）

19 条因果边里， **13 条**只由函数调用顺序或端口先后决定，代码里没有一句话表达这个意图

每类指令发射前只**检查**一个部件，却**占用**好几个：单向挡住，读代码时很容易看漏

注释说乘除与第二 ALU 是同一部件，代码行为并非如此

模拟器有部件也有指令，但“**哪条指令在哪一拍、经过哪个部件、挡住了谁**”没有写成可检查的规格

例子： CVA6 模型里“谁挡住了谁”藏在两段代码的组合里

双发射（每拍最多发两条），实验 2 的真实记录；两段程序只差指令顺序；右侧方框是发射的拍

情况 A： 先加载，后分支

#8	lw （加载）	第 4 拍
#9	bne （分支）	第 4 拍

同一拍都发出去了；这一拍“存储单元”上有两个占用者

情况 B： 先分支，后存储

#8	bne （分支）	第 4 拍
#9	sw （存储）	第 5 拍

记录：第 4 拍被“存储单元”挡住，挡住它的是第 8 条（分支）

```
def issue_branch(self):    #  分支发射时
    self.alu = True
    self.branch = True
    # Stores are not allowed yet
    self.stu = True          #  也标“存储单元”忙

def issue_ldu(self):       #  加载发射时
    self.ldu = True
    self.stu = True          #  也标“存储单元”忙
```

发射**前**：每类只查自己那一位——存储查“存储单元”，分支只查“分支单元”

model.py 第 290–300 、 323–335 行

1 挡住是单向的

分支挡得住存储；加载、存储都挡不住分支

2 意图看不出

唯一的说明是一句注释“ Stores are not allowed yet”

IR 里明确写成：分支对存储单元“**只占用、不检查**”，存储对它“**检查并占用**”

背景 ②：把微架构明确写出来，AI 就写得更好

1 HINT （arXiv 2608.07625）的启发

在规格与 RTL 之间加一层可执行的中间层，写明状态归属、提交纪律；AI 先写这一层，再照着写 RTL

2 Representation Bottleneck （arXiv 2604.17097）

6 种中间表示、 202 个任务：换表示，仿真通过率 **3% → 88%**；换更强的模型，提升不到 1.25 倍

3 FML （合作者）

按部件建模，读别的部件时必须写明读本拍值还是上一拍值——和中间 IR 一样，体现的是实际微架构

我们比 HINT 多了什么

HINT 是**事务级**，原文明确不做周期精确；我们写明本拍还是下一拍

HINT 没有容器面 / 状态面和因果边；出错时只能说“错在哪一层”

我们能追到**哪个部件、哪一面、哪一拍、被谁挡住**

目的：让 AI 在**复杂处理器**上也能生成 RTL，并且**更快、更好**——复杂处理器必须处理指令与结构之间的关系

问题：通过测试的第一版，并不可控

可控的标准	含义	curryGPU 第一版的真实情况
结构一致	RTL 的级、状态、连线与设计一致	功能被放到了图上没有的流水级，功能测试全过
依赖显式	每条跨级的时序依赖都写下来	队列深度改 1 → 2，打破了没写下来的“发射间隔 ⩾ 2”
规格可信	每条规格有来源、经过独立核对	一句规格来自 AI 写的核实报告，本身是错的，验收仍判通过
修改可预测	改一处之前知道会影响哪里	某条路径从 6 拍变 9 拍，慢 **6–7%**，人试了几轮才找到

四个问题的共同点：测试只看最终结果，**看不到结构与结构之间的因果**

AI 自己修正时，缺的是“错从哪里来”

给 AI 的反馈	例：队列深度改动后出错	例：两个模块接起来出错	AI 可能怎么做
测试结果	“第 256 号输出错了”	“整合后第 5051 拍不一致”	在报错处打补丁
第一处分歧	“下游某状态是旧的”	“下游模块的输入不一致”	去改没错的下游模块
按模块归因（ChatCPU）	归到后改的模块	归到后放进去的模块（其实两者都没错）	改错模块
带因果的 IR	**改动打破了“发射间隔 ≥ 2”，依赖方是下游推进**	**这条同拍的边在接口上没导出**	**改对地方**

前三种反馈都把 AI 引向**报错的位置**；只有沿因果关系，才能指向**出错的来源**

核心思想：加入因果，把结构拆成容器面与状态面

容器面：占位置
有限的位置，被占了别人就得等
例： scoreboard 最多 8 条；功能单元一拍只接一条
追问：现在是谁占着？什么时候走？

状态面：存信息
被写、被读，影响后面的决定，但不让人排队
例：寄存器是否准备好；分支历史表
追问：上一次是谁、哪一拍改的？

1 一个结构可以两面都有

scoreboard 既有 8 个位置（容器面），又记着每个寄存器还有谁没写完（状态面）；一次提交同时改变两面

2 指令是在结构里流动的“灵魂”

每类指令按路线依次使用各个结构；用五项规矩写清怎么用：几个位置、占多久、在哪判断、知道什么、多人抢时给谁

硬件 = **指令在使用结构做事**；把这层关系写清楚， AI 才能在复杂处理器上写对

因果边：一个动作经过某一面影响另一个动作

每条边标明“本拍就看到”还是“下一拍才看到”。例：一条乘法后紧跟一条用它结果的加法

	第 t 拍	第 t+1 拍	第 t+2 拍	第 t+3 拍
mul a0, s0, s1	发射	执行中	算完	提交
add a1, a0, s2		等待	发射	算完
功能单元（容器面）		被乘法占着		
a0 是否就绪（状态面）		没有	本拍算完即可用	

1　加法的一次等待，记下两个原因、各自“被谁挡住”

容器面：功能单元被乘法占着；状态面： a0 的写入者是那条乘法，还没算完

2　“本拍算完、本拍可用”在模拟器代码里没写

只是函数调用顺序带出来的——这类隐含的因果，正是 IR 要显式写出来的

模拟器按功能组织代码，因果**藏在调用顺序里**； IR 把它变成**可检查、可追溯**的边

用途一：让 AI 在复杂处理器上也能生成 RTL，更快、更好



1 为什么复杂处理器需要它

指令能否前进取决于它和各个结构的关系；只给功能描述， AI 只能试错

2 给 AI 的不再只是“功能描述”

而是每个结构的两面、谁检查谁占用、哪些本拍可见、哪些要等下一拍

3 本拍 / 下一拍正好对应 RTL

隔一拍的边 → 读触发器；同拍的边 → 读组合逻辑输出

4 最难的地方被显式化

跨级同拍依赖、接口时序，是 AI 整合多个模块时最常出错的地方

证据（初步）

route B： 两模块各自全对、整合后出错，原因正是接口缺了一条同拍边

待做：在 CVA6 上比较有 / 无因果 IR 时 AI 的生成成功率与迭代次数——“更快、更好”要由它证明

一句话：**时序由模型规定，实现由 AI 完成，正确性由因果 IR 检查**

用途二：可追溯的 AI 验证

沿因果边追到的来源	AI 怎么做
① 某个结构内部实现错了	修
② 结构之间的接口 / 同拍与寄存弄反了	修
③ 改动打破了别人依赖的时序规矩	修，或登记规矩的修改
④ 规格或参考本身错了	**停下，报告给人**
⑤ IR 漏画了一条依赖	**停下，报告给人**
⑥ 追不下去	**停下，报告给人**

为什么要“报”

规格错时让 AI 先试着修，它多半能凑出一个“通过”

给 AI 一个合法的“停下”出口，作弊大幅减少（ImpossibleBench： 54% → 9%）

IR 、检查对 AI 只读

可追溯：每次失败都给出**哪个结构、哪一面、哪条边、哪一拍、被谁挡住**

和已有工作的关系

工作	有什么	缺什么（我们补的）
HINT （2608.07625）	规格与 RTL 之间的可执行中间层；状态归属	不做周期精确；报错只到“哪一层”
ChatCPU （DAC'24）	周期精确模型作参考，逐模块生成	每模块一个延迟；参考与 RTL 同源；归因到“新模块”
Kôika / BSV / Assassyn	同拍 / 寄存是语言概念，直接编译	用于设计，不用于检查 AI 写的 RTL、不做归因
基于模型的诊断（Reiter'87）	先有模型，再用矛盾找故障部件	连线默认不出错；没有规格条目这一类
Zeller 因果链（FSE'02）	对比两次运行找原因	没有内部规格，要靠大量重跑

我们的位置：**容器面 / 状态面 + 周期级因果边**，同时作为 AI 生成的规格和可追溯验证的依据

验证这个思想：在 CVA6 上从模拟器得到因果 IR

1　平台： CVA6

开源 RISC-V，官方带 Python 周期级性能模型和可综合 RTL

2　只看模型、逐条数出 IR

14 个结构（10 个有容器面、5 个有状态面）、19 条因果边、10 种等待原因

3　三方向检查

没漏画 · 每次等待能解释 · 只用 IR 的规则能预测模型

1,312

次运行，0 次解释不了的等待

19 / 19

条因果边都被触发过

100%

12.5 万条指令的发射拍，只用 IR 就全部推对

结论：容器面 / 状态面 + 因果边的写法，**足以完整描述**这个模拟器的时序行为

这套写法也会暴露自己的问题： IR 被修正了 7 次

1 预测错误的惩罚可以中途变短

状态面（分支历史）在等待期间被提交更新，预测改判——两条边碰在一起才出现

2 “检查”与“占用”要分开

每类指令发射前只检查一个单元，却占用好几个：分支挡得住存储，存储挡不住分支——单向挡住

3 指令队列的对齐规则没写清

预测器 307 个错误由此而来，补上后全部消失

这三次修正都来自“**只用 IR 去预测**”：能解释每一步，不等于 IR 的说法是对的

下一步

1　按 IR 设计 FML 模型

每个结构一个模块，状态分寄存器 / 本拍信号，隐含顺序写成显式信号；设计初稿已完成

2　用官方 RTL 核对 IR

先查 13 条只由代码顺序决定的边：真实硬件是不是也这样

3　第一次 AI 生成实验

CVA6 控制通路：有 / 无因果 IR 时， AI 生成的成功率、迭代次数、假修复

4　可追溯验证实验

注入错误，看能否归到正确的来源与类别；与 Zeller 式、 SAT 调试式的方法对照

两条线最终汇合：**同一份因果 IR ，既是 AI 生成的规格，也是验证追溯的地图**

想和大家讨论的问题

1 **容器面 / 状态面的划分合理吗?**

一个结构可以两面都有；分开是为了追因果时往不同方向追。有没有它覆盖不了的硬件结构?

2 **为什么不直接编译?**

时序已写清， BSV / Kôika 可直接编译出 RTL 。我们强调抽象、实现选择与检查归因，够不够?

3 **下一个平台**

CVA6 没有多请求方仲裁和 credit 。下一个用 Vortex 还是 curryGPU ?